

Author	Reviewer	Approver
BaoHT5		

Workshop 2: Building Real-World Chatbot Systems Using Azure OpenAI API

Workshop Objective

Enable teams of 5 participants to collaboratively design, implement, and test advanced multi-turn chatbot systems powered solely by the OpenAI SDK. The focus is on solving real-life or company problems by generating and using mock data, leveraging OpenAI's chat completion features, function calling, batching, conversation message management, and sophisticated prompting techniques such as few-shot and chain-of-thought reasoning.

Open-ended Question

As a team, brainstorm and build a chatbot application that addresses a concrete real-life or organizational challenge (e.g., customer service automation, internal HR assistant, technical support, finance advisory). Generate relevant mock data as input to your LLM prompts to simulate realistic user queries or data sources. Use OpenAI SDK features exclusively—such as function calling for dynamic data retrieval, batching requests for efficiency, and detailed conversation message management—to implement a robust, context-aware chatbot system.

Applications (Choose One or Propose Your Own)

- Customer Support Automation
- Employee Onboarding Assistant
- Technical Support / IT Helpdesk Bot
- Expense Reporting Assistant
- Sales Lead Qualification Chatbot

AI Tools Usage Guidelines

Tool	Usage
OpenAI Python SDK	Use chat completion function calling, batching, message management, and output formatting techniques to build the chatbot backend logic and prompt engineering.

Teamwork Approach

- All members collaboratively contribute to **all parts** of the system: ideation, prompt creation, API call coding, conversation flow management, and UI prototyping (CLI or minimal web).
- Use a **shared Jupyter notebook** or simple UI environment for development.
- Emphasize iterative prompt refinement, handling edge cases, and managing conversation context properly.
- Generate your own **mock datasets** representing realistic inputs and knowledge bases for the chatbot.

Agenda

1. **Identify Real-Life Problem & Generate Mock Data**
Select a domain with a clear, practical problem. Generate mock user queries, knowledge base snippets, or data tables to embed in prompts.
2. **Design Conversation Flows & Prompt Templates**
Collaboratively design few-shot and chain-of-thought prompt templates incorporating your mock data.
3. **Implement OpenAI SDK Calls**
Code chat completions with conversation message arrays, enable function calling to handle dynamic queries, and batch requests where appropriate.
4. **Build Interaction Interface**
Create a CLI or simple UI to interact with your chatbot, displaying multi-turn conversations with context preserved.
5. **Test, Debug & Refine Prompts**
Run diverse test cases including ambiguous inputs and edge scenarios. Optimize prompt structure and API usage.
6. **Demo & Team Reflection**
Present your chatbot's capabilities, challenges encountered, and how you overcame them.

Deliverables by End of Workshop

- Clearly defined real-world problem and mock data schema.
 - Well-crafted prompt templates using few-shot and chain-of-thought techniques with mock data.
 - Complete OpenAI SDK-based chatbot code implementing chat completion, function calling, batching, and message management.
 - Tested conversation logs demonstrating multi-turn dialogue with context.
 - Team presentation with demo, insights, and lessons learned.
-

Detailed Example: Expense Reporting Assistant Chatbot

Problem Statement

Employees find expense reporting complex and time-consuming, leading to errors and delays. The chatbot should guide employees to submit correct expense reports, answer policy FAQs, and calculate reimbursements dynamically.

Mock Data Examples

```
# Sample company expense policy FAQ snippets
expense_policy_faqs = [
    "Receipts are required for all expenses above $25.",
    "Travel expenses must be approved before booking.",
    "Meal expenses are reimbursed up to $50 per day.",
    "Taxi expenses require detailed trip logs.",
]

# Sample mock expense report entries
mock_expense_reports = [
    {"employee_id": "E123", "date": "2025-05-01", "category": "Meals",
    "amount": 45.00},
    {"employee_id": "E123", "date": "2025-05-02", "category": "Taxi",
    "amount": 30.50},
    {"employee_id": "E456", "date": "2025-05-01", "category": "Travel",
    "amount": 1200.00},
]
```

Example Function to Calculate Reimbursement (to use with OpenAI function calling)

```
def calculate_reimbursement(expenses):
    # Simplified reimbursement calculation with policy constraints
    total = 0
    for e in expenses:
        if e['category'] == 'Meals':
            reimbursed = min(e['amount'], 50)
        elif e['category'] == 'Taxi':
            reimbursed = e['amount'] # fully reimbursed
        elif e['category'] == 'Travel':
            reimbursed = e['amount'] # assuming approved
        else:
            reimbursed = 0
        total += reimbursed
    return f"Total reimbursement amount is ${total:.2f}"
```

Sample OpenAI SDK Usage (Python snippet)

```

import openai
import json
from openai import AzureOpenAI
import os
import json

# Initialize OpenAI client
client = AzureOpenAI(
    api_version="2024-07-01-preview",
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
)

# Mock function to calculate reimbursement
def calculate_reimbursement(expenses):
    total = 0
    for e in expenses:
        if e['category'].lower() == 'meals':
            reimbursed = min(e['amount'], 50)
        elif e['category'].lower() == 'taxi':
            reimbursed = e['amount']
        elif e['category'].lower() == 'travel':
            reimbursed = e['amount']
        else:
            reimbursed = 0
        total += reimbursed
    return f"Total reimbursement amount is ${total:.2f}"

# Few-shot conversation messages
messages = [
    {"role": "system", "content": "You are a helpful expense reporting assistant."},
    {"role": "user", "content": "What is the meal expense limit?"},
    {"role": "assistant", "content": "Meal expenses are reimbursed up to $50 per day."},
    {"role": "user", "content": "Please calculate reimbursement for my expenses."},
]

# Define the function for OpenAI to call
functions = [
    {
        "name": "calculate_reimbursement",
        "description": "Calculate total reimbursement amount from expense list",
        "parameters": {
            "type": "object",
            "properties": {
                "expenses": {
                    "type": "array",
                    "items": {
                        "type": "object",
                        "properties": {
                            "category": {"type": "string"},
                            "amount": {"type": "number"}
                        }
                    }
                }
            }
        }
    }
]

```

```

        "required": ["category", "amount"]
    }
    },
    "required": ["expenses"]
}
]

# User expenses mock data
user_expenses = [
    {"category": "Meals", "amount": 45},
    {"category": "Taxi", "amount": 30.5},
    {"category": "Travel", "amount": 1200},
]

# Append user prompt with expenses as stringified JSON
user_content = f"Calculate reimbursement for these expenses: {json.dumps(user_expenses)}"
messages.append({"role": "user", "content": user_content})

# Call OpenAI chat completion with function calling enabled
completion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    functions=functions,
    function_call="auto", # let the model decide if function call is
    needed
)

response_message = completion.choices[0].message

if response_message.get("function_call"):
    # Parse function call details
    func_name = response_message["function_call"]["name"]
    func_args = response_message["function_call"]["arguments"]

    if func_name == "calculate_reimbursement":
        result = calculate_reimbursement(func_args["expenses"])
        print("Function call result:", result)
    else:
        print("Chatbot reply:", response_message["content"])

```

Suggested Testing Scenarios

Test ID	Scenario
TC_01	Ask chatbot about expense policies → accurate, policy-based reply
TC_02	Submit sample expense entries and trigger reimbursement calc
TC_03	Handle ambiguous or incomplete expense details gracefully
TC_04	Multi-turn conversation remembers previous inputs and context

References for Workshop

- **OpenAI Python SDK** (ChatCompletion, function calling, batching, conversation message management)
<https://platform.openai.com/docs/api-reference/chat/create>
- **Prompting Techniques** (Few-shot, Chain-of-Thought)
<https://platform.openai.com/docs/guides/chat/prompt-design>
- **OpenAI Function Calling**
<https://platform.openai.com/docs/guides/gpt/function-calling>